Ministry of Higher Education
and Scientific Research

International Institute of Technology of
Sfax

وزارة التعليم العالي والبحث العلمي

المدرسة العليا الدولية الخاصة للتكنولوجيا بصفا قس

Final Year Project

Carried out at

Presented to

The International Institute of Technology of Sfax

In partial fulfillment of the requirements for the

National Engineering Degree in
Computer Science Engineering

By

Youssef AYADI

Smart Internship and Project Management System (SIPMS — CLINISYS Platform)

Defended on / / 2026, before the examination committee :

Mr. Fahmi Kallel	Committee President
.....	Reviewer
Mrs. Rahma Bouaziz	Academic Supervisor
Mr. Marwen Hadrich	Industrial Supervisor

Academic Year : 2025 / 2026

To my beloved parents,

whose unconditional love, endless sacrifices,
and unwavering belief in me have been
the greatest source of strength throughout my journey.

To my family and friends,
for their constant encouragement and support.

To all my teachers and mentors
who have guided me with knowledge and wisdom.

This work is dedicated to you all.

Youssef Ayadi

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to **Allah** for granting me the strength, patience, and perseverance to complete this work.

I would like to express my deepest gratitude to my academic supervisor, **Mrs. Rahma Bouaziz**, for her invaluable guidance, constructive feedback, and continuous support. I am also profoundly thankful to my industrial supervisor, **Mr. Marwen Hadrich**, for his mentorship, expertise, and for welcoming me into **CLINISYS**.

I would also like to extend my sincere thanks to the Committee President, **Mr. Fahmi Kallel**, and to the honorable Reviewer for accepting to evaluate this work. Their insights and expertise are highly appreciated.

I extend my sincere thanks to the entire team at **CLINISYS** for welcoming me into their organization and providing an enriching and stimulating professional environment. Their collaboration and trust were essential to the success of this project.

My heartfelt appreciation goes to the faculty and staff of **IIT Sfax** for the quality education and technical training they have provided throughout my studies. The knowledge and skills acquired during my academic journey have been the foundation upon which this project was built.

I am also deeply grateful to my family, whose love, encouragement, and sacrifices have sustained me throughout my academic career. A special thank you to my friends and colleagues for their moral support and camaraderie.

Finally, I wish to thank all those who, directly or indirectly, contributed to the completion of this project.

Youssef Ayadi

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JPA	Java Persistence API
ORM	Object-Relational Mapping
MVC	Model-View-Controller
SPA	Single-Page Application
SQL	Structured Query Language
SMTP	Simple Mail Transfer Protocol
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
UI	User Interface
UX	User Experience
SIPMS	Smart Internship and Project Management System
MCQ	Multiple-Choice Question
CIN	National Identity Card Number
CV	Curriculum Vitae
PFE	Final Year Project (Projet de Fin d'Études)
IDE	Integrated Development Environment
BCrypt	Blowfish Crypt (password hashing algorithm)
KPI	Key Performance Indicator

Abbreviations used in this report

GENERAL INTRODUCTION

In today's digitally driven world, organizations are under constant pressure to streamline their internal processes and improve operational efficiency. Among the administrative challenges faced by technology companies, the management of internship candidates stands out as a particularly complex and resource-intensive task. When handled manually, it involves paper-based registration, unstructured document storage, time-consuming evaluations, and error-prone communication workflows.

It is within this context that our final-year project was born. Carried out at **CLINISYS**, a company specializing in advanced technological solutions, this project aims to design and develop the **Smart Internship and Project Management System (SIPMS)** — a full-stack web platform that digitizes and automates the complete lifecycle of an internship candidate : from their physical registration at the reception desk, through automated competency testing via customized quizzes, to their final assignment to a supervised project.

The SIPMS platform is built on a modern, robust technology stack : a reactive frontend powered by **React (Vite)**, a secure backend built with **Spring Boot (Java 17)**, a relational database using **MySQL**, and a stateless authentication system based on **JSON Web Tokens (JWT)**. It further incorporates artificial intelligence features for project ranking and candidate-to-supervisor matching, alongside automated email notifications and multi-role dashboards.

This report is organized into four main chapters :

- **Chapter 1 — General Project Framework :** Presentation of the host organization, analysis of the existing system, problem statement, proposed solution, and development methodology.
- **Chapter 2 — Requirements Analysis and Specification :** Identification of actors, functional and non-functional requirements, and use case modeling.

— **Chapter 3 — System Design :** Overall architecture, class diagrams, sequence diagrams, and database schema.

— **Chapter 4 — Implementation :** Development environment, Scrum sprint execution, and presentation of the realized user interfaces.

We conclude this report with a summary of the achieved results and future enhancement perspectives for the SIPMS platform.

Chapitre

1 GENERAL FRAMEWORK

1.1 INTRODUCTION

This chapter presents the general framework of our final-year project. We begin by introducing the host organization CLINISYS, followed by a critical analysis of the existing system. We then define the core problem statement, present the proposed SIPMS solution, and describe the Scrum agile methodology adopted for the development process.

1.2 PRESENTATION OF THE HOST ORGANIZATION

1.2.1 About CLINISYS

CLINISYS is a technology company specializing in the development of innovative digital solutions for industrial and academic sectors. With deep expertise in software engineering, the company supports its clients in their digital transformation journeys by delivering custom-built systems that combine performance, security, and usability.

As part of its commitment to talent development and innovation, CLINISYS welcomes a growing number of interns from higher education institutions each year. This ongoing engagement with academic talent has highlighted the urgent need for a structured digital tool capable of managing the full internship lifecycle efficiently.

1.2.2 Key Activity Areas

- Custom web and mobile application development
- Digital transformation consulting and process optimization
- Artificial intelligence and data analytics integration
- Enterprise information system management and hosting

1.3 ANALYSIS OF THE EXISTING SYSTEM

1.3.1 Description of the Current System

Prior to the development of SIPMS, CLINISYS managed its internship and project workflows using entirely manual processes. Candidate registration was performed on paper forms, communication relied on unstructured emails, and progress tracking depended on Excel spreadsheets maintained manually by reception staff. Document management — including CVs, cover letters, and academic transcripts — was disorganized and difficult to query.

1.3.2 Identified Limitations

The analysis of the existing system revealed several critical dysfunctions, summarized in the table below :

Identified Problem	Organizational Impact
Manual candidate registration	High risk of data entry errors, loss of records, slow onboarding process
No competency evaluation system	Inability to objectively assess candidates' technical skills
Non-automated communication	Significant delays in sending login credentials and notifications
No year-based filtering	Difficulty retrieving and managing candidates from a given intake year
Absent document management	CVs and supporting documents stored in an unstructured manner
Manual project assignment	Subjective, time-consuming process with no defined relevance criteria

TABLE 1.1 – Summary of existing system limitations

1.4 PROBLEM STATEMENT

Given the shortcomings of the current system, CLINISYS expressed the need for a centralized digital solution capable of handling the complete internship candidate lifecycle. The core problem statement is formulated as follows :

"How can a smart web platform be designed and developed to automate and centralize the management of internship candidates — from registration to supervised project assignment — while ensuring data security, action traceability, and an optimal user experience for all stakeholders involved ?"

1.5 PROPOSED SOLUTION

1.5.1 The SIPMS Platform

In response to this problem, we propose the development of the **Smart Internship and Project Management System (SIPMS)**, a full-stack, multi-role web platform. The solution provides :

- A **Reception Panel** enabling receptionists to register candidates, upload their documents (CV, cover letter, academic transcript, ID card), and filter candidates by internship year.
- An **automated quiz system** for objective, role-specific evaluation of candidates' technical competencies.
- An **AI engine** for project ranking and intelligent matching between candidates and available supervisors.
- An **automated email system** that instantly sends login credentials upon candidate account creation.
- **Role-differentiated dashboards** tailored to each user type : Administrator, Manager, Receptionist, Candidate, and Supervisor.
- **Complete application lifecycle management**, tracking each candidature from submission through review, quiz evaluation, and final project assignment.

1.5.2 Technology Stack Overview

Layer	Technology	Role
Frontend	React (Vite) + CSS	Reactive single-page application
Backend	Spring Boot (Java 17)	RESTful API, business logic
Security	Spring Security + JWT	Stateless authentication, RBAC
Database	MySQL 8	Relational data persistence
Email	JavaMail (Gmail SMTP)	Automated transactional emails
File Storage	Spring Multipart	CV and document upload
AI Module	Custom scoring engine	Project ranking, candidate matching

TABLE 1.2 – SIPMS technology stack

1.6 DEVELOPMENT METHODOLOGY

1.6.1 Choice of Methodology : Scrum

We adopted the **Scrum** agile framework for this project. This choice is justified by the iterative nature of the requirements, the need to deliver functional increments regularly, and the flexibility required to accommodate changes during development.

1.6.2 Scrum Framework

Scrum organizes development into short iterations called **Sprints**, typically lasting two to three weeks. Each sprint produces a potentially shippable product increment. The key Scrum artifacts used in this project are :

- **Product Backlog** : A prioritized list of all features to be developed.
- **Sprint Backlog** : The subset of the Product Backlog selected for a given sprint.
- **Increment** : A functional version of the software delivered at the end of each sprint.

Sprint	Main Objective	Delivered Features
Sprint 1	Authentication & User Management	JWT login, RBAC, role management, base dashboards, change password
Sprint 2	Reception Module	Candidate registration, year filtering, document upload, automated welcome email
Sprint 3	Quiz Module	Quiz creation, candidate assignment, quiz interface, scoring, results
Sprint 4	AI & Assignment Module	Project ranking, candidate/supervisor matching, final assignment workflow

TABLE 1.3 – SIPMS sprint planning

1.7 CONCLUSION

This chapter established the organizational context of the SIPMS project, highlighted the critical limitations of the manual existing system, and presented the proposed digital solution along with its development methodology. The following chapter will provide a detailed analysis of the functional and non-functional requirements of the platform.

Chapitre

2 REQUIREMENTS ANALYSIS AND SPECIFICATION

2.1 INTRODUCTION

This chapter formalizes the requirements of the SIPMS platform. We identify all system actors, enumerate functional and non-functional requirements, and model system behavior using UML use case diagrams.

2.2 SYSTEM ACTORS

Actor	Description
Administrator	Full system control : manages all users, roles, configurations, and audit logs.
Manager	Oversees application lifecycle, assigns supervisors, and monitors AI rankings.
Receptionist	Registers candidates, uploads documents, filters by year/specialty, assigns quizzes.
Candidate	Completes profile, takes quizzes, submits applications, tracks status.
Supervisor	Views assigned interns and coordinates project progress with the manager.

TABLE 2.1 – System actors and their roles

2.3 FUNCTIONAL REQUIREMENTS

2.3.1 Authentication and Access Control

- Users authenticate via email/password using stateless JWT tokens.
- Role-Based Access Control (RBAC) restricts features based on assigned role.
- New candidates must change their temporary password on first login.
- Automated welcome emails with credentials are sent upon account creation.

2.3.2 Reception and Candidate Management

- Receptionists register candidates with personal info (name, email, phone, CIN, specialty, internship year).

- Multiple documents are uploadable per candidate : CV, cover letter, transcript, ID card.
- Candidates are filterable by year and specialty in the reception panel.
- A quiz is automatically assigned to each newly registered candidate.

2.3.3 Quiz and Evaluation

- Admins create quizzes with multiple-choice questions and configurable time limits.
- A countdown timer is displayed during the quiz ; auto-submission occurs on expiry.
- Scores are automatically calculated and candidate status updated accordingly.

2.3.4 Application Lifecycle

- Candidates submit online applications linked to available projects.
- Application statuses follow : Pending $\rightarrow$ Under Review $\rightarrow$ Accepted / Rejected.
- Managers update statuses and assign supervisors to accepted applications.

2.3.5 AI Module

- AI ranks available projects by relevance and scores candidates.
- Candidate-to-supervisor matching recommends the best-fit supervisor by specialty and availability.
- Rankings are persisted and viewable by managers.

2.4 NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Security	JWT authentication, BCrypt password hashing, `@PreAuthorize` on all endpoints.
Performance	API responses under 2 seconds ; lazy-loading on the frontend.
Usability	Intuitive, responsive, role-differentiated dashboards requiring no technical expertise.
Reliability	Non-blocking email and file upload operations ; fault-tolerant error handling.
Maintainability	Layered architecture : Controller $\rightarrow$ Service $\rightarrow$ Repository ; reusable components.
Scalability	Architecture supports adding new modules and roles without breaking existing features.

TABLE 2.2 – Non-functional requirements of SIPMS

2.5 USE CASE MODELING

FIGURE 2.1 – Global use case diagram — SIPMS platform

Field	Description
Use Case	Register a New Candidate
Actor	Receptionist
Precondition	Receptionist is authenticated with RECEPTIONIST role
Main Scenario	1. Click “New Candidate” 2. Fill registration form 3. Upload documents 4. Submit 5. System creates account, generates password, sends email, assigns quiz
Postcondition	Account created ; email sent ; quiz assigned
Exception	Duplicate email triggers error message

TABLE 2.3 – Use case : Register a new candidate

Field	Description
Use Case	Take a Quiz
Actor	Candidate
Precondition	Candidate is authenticated and a quiz has been assigned
Main Scenario	1. Navigate to Quiz page 2. Start quiz with timer 3. Answer MCQ questions 4. Submit before timer expires 5. Score is calculated and status updated
Postcondition	Score recorded ; status updated
Exception	Timer expiry triggers automatic submission

TABLE 2.4 – Use case : Take a quiz

2.6 CONCLUSION

This chapter provided a complete specification of the SIPMS platform requirements. The actor identification, structured requirements, and use case models serve as the design foundation addressed in the next chapter.

Chapitre

3 SYSTEM DESIGN

Sommaire

3.1 INTRODUCTION

This chapter presents the technical design of the SIPMS platform. We detail the overall system architecture, the class model, the sequence diagrams for key workflows, and the relational database schema.

3.2 GENERAL ARCHITECTURE

3.2.1 Architectural Pattern

SIPMS follows a **client-server architecture** with a clear separation of concerns across three tiers :

- **Presentation Layer** : A React (Vite) single-page application running in the browser, communicating with the backend exclusively via RESTful HTTP requests.
- **Business Logic Layer** : A Spring Boot application exposing a secured REST API, organized into Controllers, Services, and Repositories following the layered architecture pattern.
- **Data Layer** : A MySQL 8 relational database accessed via Spring Data JPA (Hibernate), with schema managed through entity definitions.

FIGURE 3.1 – Three-tier architecture of the SIPMS platform

3.2.2 Backend Package Structure

The Spring Boot backend is organized into the following packages :

Package	Responsibility
`controller`	Exposes REST endpoints ; handles HTTP requests and responses
`service`	Contains business logic ; orchestrates between controllers and repositories
`repository`	Spring Data JPA interfaces for database access
`entity`	JPA entity classes mapped to database tables
`dto`	Data Transfer Objects for request/response payloads
`security`	JWT filter, authentication entry point, Spring Security configuration
`common`	Shared utilities : ApiResponse wrapper, FileStorageService, AuditService
`ai`	AI scoring and matching engine

TABLE 3.1 – Backend package structure

3.2.3 Security Architecture

Authentication in SIPMS is stateless and JWT-based. The security flow operates as follows :

1. The client sends credentials (email + password) to `POST /api/auth/login`.
2. Spring Security validates credentials via `UserDetailsService`.
3. On success, a signed JWT token is generated and returned to the client.
4. For subsequent requests, the client includes the token in the `Authorization: Bearer` header.
5. The `JwtAuthenticationFilter` intercepts every request, validates the token, and populates the `SecurityContext`.
6. `@PreAuthorize` annotations on controller methods enforce role-based access.

3.3 CLASS DIAGRAM

3.3.1 Core Entity Relationships

The main entities of the SIPMS domain model and their relationships are described below :

Entity	Key Attributes	Relationships
User	id, firstName, lastName, email, passwordHash, specialty, internshipYear, status, active	ManyToMany → Role
Role	id, name (ROLE_ADMIN, etc.)	ManyToMany ← User
Application	id, intakeMethod, status, managerNotes, createdAt	ManyToOne → User (candidate, registeredBy), Supervisor
Document	id, fileName, filePath, fileType, fileSize, documentType	ManyToOne → Application, User
Quiz	id, title, specialty, duration, active	OneToMany → QuizQuestion
QuizQuestion	id, questionText, correctAnswer, options	ManyToOne → Quiz
QuizAttempt	id, score, completedAt, answers	ManyToOne → User, Quiz
Supervisor	id, firstName, lastName, specialty, currentInterns, active	OneToMany ← Application
Project	id, title, description, status, company	ManyToOne → Supervisor
Notification	id, title, message, read, createdAt	ManyToOne → User

TABLE 3.2 – Main entities and their relationships

FIGURE 3.2 – Simplified class diagram — SIPMS core entities

3.4 SEQUENCE DIAGRAMS

3.4.1 Sequence : Candidate Registration with Document Upload

This sequence describes the complete candidate registration flow performed by the receptionist :

1. Receptionist submits the registration form via the React frontend.
2. `POST /api/users` is called with candidate data including `internshipYear`.
3. `UserService.createUser()` generates a temporary password, encodes it with BCrypt, creates the user, assigns the ROLE_CANDIDATE role, and saves to the database.
4. `EmailService.sendWelcomeEmail()` sends an asynchronous HTML email with credentials.
5. If files were selected, `POST /api/candidates/{id}/documents` is called with multipart form data.
6. `FileStorageService.storeFile()` saves each file to the local filesystem and returns a relative path.
7. A `Document` entity is persisted per uploaded file, linked to the candidate's physical application.
8. The frontend refreshes the candidates table with the new entry.

FIGURE 3.3 – Sequence diagram — Candidate registration with document upload

3.4.2 Sequence : Quiz Execution

1. Candidate navigates to the Quiz page ; frontend calls `GET /api/quizzes/{id}`.
2. Backend returns quiz metadata and questions (without correct answers).
3. Frontend renders questions with a countdown timer.
4. Candidate submits answers via `POST /api/quizzes/submit`.
5. `QuizService.submitQuiz()` compares submitted answers against stored correct answers and calculates the score.
6. Candidate's `status` is updated to `quiz_completed` or `quiz_failed` based on the passing threshold.
7. Score and completion timestamp are persisted in a `QuizAttempt` record.

3.4.3 Sequence : AI-Powered Candidate Matching

1. Manager triggers matching via `POST /api/ai/match-candidates/{supervisorId}`.
2. `AiService` retrieves all active supervisors and accepted candidates.
3. Each candidate is scored against the supervisor's specialty, current intern load, and quiz scores.
4. Results are ranked and persisted as `AiRanking` records.
5. Manager views the ranked list on the AI Insights dashboard.

3.5 DATABASE SCHEMA

3.5.1 Main Tables

Table	Key Columns
`users`	id, first_name, last_name, email, password_hash, specialty, internship_year, status, active, must_change_password, created_at
`roles`	id, name
`user_roles`	user_id, role_id
`applications`	id, candidate_id, supervisor_id, registered_by, intake_method, status, manager_notes, created_at
`documents`	id, application_id, uploaded_by, file_name, file_path, file_type, file_size, document_type, created_at
`quizzes`	id, title, specialty, duration_minutes, active, created_at
`quiz_questions`	id, quiz_id, question_text, options (JSON), correct_answer, points
`quiz_attempts`	id, user_id, quiz_id, score, completed_at, answers (JSON)
`supervisors`	id, first_name, last_name, specialty, email, current_interns, active
`projects`	id, title, description, company, status, supervisor_id, created_at
`notifications`	id, user_id, title, message, is_read, created_at
`ai_rankings`	id, entity_type, entity_id, score, rank, created_at

TABLE 3.3 – SIPMS database table overview

3.6 CONCLUSION

This chapter presented the complete technical design of SIPMS, covering the layered architecture, security model, entity relationships, key sequence diagrams, and database schema. These design decisions provide a solid blueprint for the implementation phase described in the next chapter.

Chapitre

4 IMPLEMENTATION

Sommaire

4.1 INTRODUCTION

This chapter presents the practical realization of the SIPMS platform. We first describe the development environment and tools used, then walk through the four Scrum sprints executed during the project, presenting the key user interfaces delivered in each sprint.

4.2 DEVELOPMENT ENVIRONMENT

4.2.1 Hardware Environment

Component	Specification
Processor	Intel Core i5 / i7, 2.4 GHz or higher
RAM	16 GB
Storage	512 GB SSD
Operating System	Windows 10 / 11 (64-bit)

TABLE 4.1 – Hardware development environment

4.2.2 Software Environment

Tool / Technology	Version	Purpose
Java (JDK)	17 LTS	Backend runtime
Spring Boot	3.2.x	Backend framework
Maven	3.9.x	Build and dependency management
MySQL	8.0	Relational database
Node.js	20 LTS	Frontend runtime
React (Vite)	18 / 5.x	Frontend framework
VS Code	Latest	Frontend IDE
IntelliJ IDEA	2023.x	Backend IDE
Postman	Latest	API testing
Git / GitHub	-	Version control
MySQL Workbench	8.0	Database management

TABLE 4.2 – Software development environment

4.3 SPRINT 1 — AUTHENTICATION AND USER MANAGEMENT

4.3.1 Sprint Backlog

#	User Story	Status
1	As a user, I want to log in with my email and password and receive a JWT token	Done
2	As a user, I want to be redirected to my role-specific dashboard after login	Done
3	As a new user, I want to be prompted to change my temporary password on first login	Done
4	As an admin, I want to create, view, edit, and delete user accounts	Done
5	As a system, I must enforce RBAC so that each route is accessible only to authorized roles	Done

TABLE 4.3 – Sprint 1 backlog

4.3.2 Key Technical Implementations

- **JwtTokenProvider** : Generates and validates tokens using a configurable secret key and expiry duration.
- **JwtAuthenticationFilter** : Intercepts all requests, extracts and validates the Bearer token, and populates the Spring Security context.
- **SecurityConfig** : Defines the HTTP security rules, CORS configuration, and public endpoints (`/api/auth/**`).
- **RoleBasedRedirect** : A React component that reads the user's role from the JWT and redirects to the appropriate dashboard path.
- **ProtectedRoute** : Guards frontend routes by checking authentication state and role membership.

4.3.3 Delivered Interfaces

FIGURE 4.1 – Login page — SIPMS CLINISYS branding

FIGURE 4.2 – Administrator dashboard — Users management panel

4.4 SPRINT 2 — RECEPTION MODULE

4.4.1 Sprint Backlog

#	User Story	Status
6	As a receptionist, I want to register a new candidate with their personal information	Done
7	As a receptionist, I want to upload supporting documents (CV, cover letter, transcript, ID) for each candidate	Done
8	As a receptionist, I want to filter the candidates list by internship year	Done
9	As a receptionist, I want to filter candidates by specialty	Done
10	As a system, I want to automatically send a welcome email with temporary credentials upon candidate creation	Done
11	As a system, I want to automatically assign an available quiz to each newly registered candidate	Done

TABLE 4.4 – Sprint 2 backlog

4.4.2 Key Technical Implementations

- **internshipYear field** : Added to the `User` entity and `UserDto`, persisted in the `users` table as an integer column.
- **Document upload endpoint** : `POST /api/candidates/{userId}/documents` accepts multipart files (cv, coverLetter, transcript, idCard, other), stores them via `FileStorageService`, and persists `Document` entities linked to the candidate's physical application.
- **Email automation** : `EmailService.sendWelcomeEmail()` sends an HTML-formatted email with login credentials from the configured Gmail SMTP account (`ayadi9732@gmail.com`) using a 16-character App Password.
- **Year filter** : The frontend `ReceptionPanel.jsx` filters the candidate list using both `internshipYear` (when set) and the creation year as a fallback.

4.4.3 Delivered Interfaces

FIGURE 4.3 – Reception Panel — Candidate list with year filter

FIGURE 4.4 – New Candidate registration modal with document upload

4.5 SPRINT 3 — QUIZ MODULE

4.5.1 Sprint Backlog

#	User Story	Status
12	As an admin, I want to create quizzes with multiple-choice questions and a time limit	Done
13	As an admin, I want to assign a quiz to a specific candidate	Done
14	As a candidate, I want to take my assigned quiz with a visible countdown timer	Done
15	As a candidate, I want to see my quiz score immediately after submission	Done
16	As a system, I want to auto-submit the quiz when the time limit is reached	Done

TABLE 4.5 – Sprint 3 backlog

4.5.2 Key Technical Implementations

- **QuizTimeLimitInterceptor** : A Spring MVC interceptor that blocks quiz submission requests that arrive after the allotted duration.
- **QuizInterface.jsx** : A React component rendering question cards with a real-time countdown timer using `useEffect` and `setInterval`.
- **Auto-submit** : When the timer reaches zero, the component automatically calls the submit handler, preventing further answer changes.
- **Score calculation** : `QuizService.submitQuiz()` iterates over submitted answers, compares them to stored correct answers, and calculates a percentage score.

FIGURE 4.5 – Quiz interface — Question view with countdown timer

4.6 SPRINT 4 — AI MODULE AND PROJECT ASSIGNMENT

4.6.1 Sprint Backlog

#	User Story	Status
17	As a manager, I want the system to rank available projects by relevance	Done
18	As a manager, I want the system to suggest the best-fit supervisor for each candidate	Done
19	As a manager, I want to view AI-generated rankings on a dedicated dashboard	Done
20	As a manager, I want to assign a supervisor to an accepted application	Done
21	As a manager, I want to conduct interviews and record results	Done

TABLE 4.6 – Sprint 4 backlog

4.6.2 Key Technical Implementations

- **AiService.rankProjects()** : Scores each available project based on supervisor capacity, project complexity, and application volume. Results are stored as `AiRanking` records.
- **AiService.matchCandidates()** : For a given supervisor, retrieves all accepted candidates and scores them by specialty alignment, quiz score, and application seniority.
- **AI Insights Dashboard** : A dedicated React page displaying sortable tables of project rankings and candidate matchings fetched from `GET /api/ai/rankings/**`.

- **Supervisor Assignment** : `PATCH /api/applications/{id}/assign-supervisor` links an accepted application to a chosen supervisor and increments their `currentInterns` counter.

FIGURE 4.6 – AI Insights page — Project rankings and candidate matching

FIGURE 4.7 – Applications management — Supervisor assignment workflow

4.7 CONCLUSION

This chapter presented the complete implementation of the SIPMS platform across four Scrum sprints. Each sprint delivered functional, tested increments that progressively built the full-featured system. The combination of a robust Spring Boot backend, a reactive React frontend, automated email workflows, and an AI-powered matching engine produces a platform that successfully addresses all the requirements identified in Chapter 2.

Chapitre

5 TESTING AND VALIDATION

Sommaire

5.1 INTRODUCTION

The purpose of testing is to ensure that the SIPMS platform meets the specified functional and non-functional requirements, operates reliably under expected loads, and maintains security against unauthorized access. This chapter presents the comprehensive testing strategy applied across all modules, the test cases executed, performance benchmarks, security validation, and a summary of results demonstrating that all requirements are satisfied.

5.2 TESTING STRATEGY

SIPMS was validated using a four-tier testing pyramid to ensure coverage at every level of the system :

5.2.1 Unit Testing

Individual components — services, controllers, utilities, and helpers — were tested in isolation using JUnit 5 and Mockito. Each method was verified for correct output given known inputs, edge cases, and error conditions. Key modules tested :

- **Authentication Service** : Login validation, token generation and parsing, role extraction, BCrypt password hashing, refresh token logic.
- **Candidate Service** : CRUD operations, file upload and path generation, quiz assignment, email trigger conditions.
- **Quiz Service** : Question scoring algorithms, attempt creation and expiration, percentage calculation, pass/fail determination.
- **Interview Service** : Scheduling validation, status transitions (SCHEDULED $\rightarrow$ COMPLETED $\rightarrow$ CANCELLED), score recording.
- **Notification Service** : Creation, retrieval, unread count, mark-as-read logic.

5.2.2 Integration Testing

All REST API endpoints were tested end-to-end using Postman collections that simulate real client requests. Each endpoint was validated against the following criteria :

- Correct HTTP status codes for success (200, 201), client errors (400, 401, 403, 404), and server errors (500).
- Request payload validation — missing required fields, invalid data types, boundary values.
- Authentication and authorization — unauthenticated requests rejected, role-based access enforced (RECEPTIONIST cannot access MANAGER endpoints).
- File upload and download flows — multipart requests, content-type validation, file size limits.
- Database transaction integrity — rollback on failure, constraint enforcement.

5.2.3 System Testing

The complete platform — frontend + backend + database — was tested as a unified system. End-to-end user journeys were executed in a staging environment mirroring production :

- Full candidate lifecycle : Registration → File Upload → Quiz Assignment → Quiz Completion → Manager Review → Acceptance.
- Cross-role workflows : Receptionist creates candidate, Manager approves and sends quiz, Candidate takes quiz, Manager interviews.
- Concurrent session handling : multiple users interacting simultaneously without data corruption.

5.2.4 User Acceptance Testing (UAT)

The platform was demonstrated to CLINISYS stakeholders (managers and receptionists) who validated each workflow against real operational scenarios. Feedback was collected and incorporated into the final sprint. All critical user stories (US-01 through US-21) were accepted.

5.3 TEST CASES

Test ID	Feature	Test Scenario	Expected Result
TC-01	Authentication	Login with valid credentials	JWT token returned, user redirected to role-based dashboard
TC-02	Authentication	Login with invalid password	401 Unauthorized, error message displayed
TC-03	Authentication	Access protected endpoint without token	403 Forbidden
TC-04	Role-Based Access	RECEPTIONIST accesses /api/candidates	200 OK, candidate list returned
TC-05	Role-Based Access	RECEPTIONIST accesses /api/admin/users	403 Forbidden
TC-06	Candidate Creation	Create candidate with valid data (firstName, lastName, email, phone, CIN)	200 OK, candidate created with ID
TC-07	Candidate Creation	Create candidate with missing email	400 Bad Request, validation error
TC-08	Internship File	Add internship file with document upload	200 OK, file stored, Document record created
TC-09	Internship File	Add internship file without document	200 OK, file record created with empty documents
TC-10	Quiz Assignment	Approve candidate and send default quiz	200 OK, User account created, email sent with credentials
TC-11	Quiz Assignment	Approve candidate with existing quizId	200 OK, existing quiz assigned to candidate
TC-12	Quiz Taking	Submit quiz with all correct answers	Score calculated, status set to QUIZ_COMPLETED
TC-13	Quiz Taking	Submit quiz after 48-hour expiry	Attempt rejected, score set to 0
TC-14	Interview	Schedule interview with valid data	200 OK, Interview created with SCHEDULED status
TC-15	Interview	Record interview result with score	200 OK, status updated to COMPLETED, score saved
TC-16	File Download	Access uploaded PDF via /uploads/... URL	200 OK, PDF served with correct Content-Type
TC-17	Email Notification	Manager approves candidate	Email sent to candidate with login credentials
TC-18	User Management	Admin creates new user account	200 OK, user created, password generated

TABLE 5.1 – Test case summary — key scenarios

5.4 PERFORMANCE TESTING

Performance testing was conducted to verify that the system meets responsiveness requirements under normal and peak loads.

5.4.1 Response Time Validation

Key API endpoints were benchmarked using Postman with 10 sequential requests. The average response time was measured and validated against the 2-second threshold :

Endpoint	Avg Response (ms)	Max (ms)	Status
POST /api/auth/login	245	412	Pass (< 2s)
GET /api/candidates	180	310	Pass (< 2s)
POST /api/candidates	320	580	Pass (< 2s)
POST /api/candidates/{id}/internship-files/with-document	850	1450	Pass (< 2s)
POST /api/quiz/submit	420	780	Pass (< 2s)
GET /api/interviews	195	340	Pass (< 2s)

TABLE 5.2 – API response time benchmarks

All endpoints responded well within the 2-second threshold, confirming that the system is performant for real-time use.

5.4.2 Load Handling

The system was tested with up to 20 concurrent simulated users performing simultaneous operations. Connection pool settings (HikariCP max 10) and thread pool configurations handled the load without timeout or data corruption.

5.5 SECURITY TESTING

Security was validated across multiple dimensions to ensure data protection and access control.

5.5.1 Role-Based Access Control (RBAC)

Each endpoint in the system is annotated with `@PreAuthorize` to enforce role restrictions. The following matrix was tested :

Endpoint Group	Admin	Manager	Receptionist	Candidate
GET /api/users		403	403	403
GET /api/candidates				403
POST /api/candidates/id/approve-and-send-quiz			403	403
POST /api/interviews			403	403
POST /api/quiz/submit	403	403	403	

TABLE 5.3 – RBAC access matrix — tested and verified

5.5.2 JWT Validation

- **Token generation** : Tokens are signed using HS512 algorithm with a 256-bit secret key and include expiration timestamp.
- **Token expiration** : Access tokens expire after 24 hours ; refresh tokens after 7 days. Expired tokens are rejected with 401.
- **Token tampering** : Modified tokens are detected during signature verification and rejected.
- **Password security** : All passwords are hashed using BCrypt with strength factor 10.

5.6 VALIDATION RESULTS

The following table maps each functional requirement to its validation status, demonstrating that all requirements are satisfied :

US ID	Requirement	Test Coverage	Status
US-01	Role-based login	TC-01 — TC-05	Verified
US-02	Admin creates users	TC-18	Verified
US-03	Password change on first login	Integration tested	Verified
US-04	Receptionist registers dossiers	TC-06 — TC-09	Verified
US-05	Admin/Manager manages users	TC-18	Verified
US-06	Supervisor management	Integration tested	Verified
US-07	Candidate submits application	Integration tested	Verified
US-08	AI ranks projects	Integration tested	Verified
US-09	AI suggests supervisor	Integration tested	Verified
US-10	Timed technical quiz	TC-12, TC-13	Verified
US-11	Auto-correct quiz	TC-12	Verified
US-12	Specialty-based quiz (48h)	TC-13	Verified
US-13	Manager sets candidate status	Integration tested	Verified
US-14	Automatic email notifications	TC-17	Verified
US-15	Role-based dashboard redirect	TC-01, TC-04, TC-05	Verified
US-16	Per-year internship files	TC-08, TC-09	Verified
US-17	View all internship files	TC-08	Verified
US-18	Approve and send quiz	TC-10, TC-11	Verified
US-19	Schedule interview	TC-14	Verified
US-20	Record interview result	TC-15	Verified
US-21	View all interviews	TC-14	Verified

TABLE 5.4 – Requirements traceability matrix — all user stories verified

5.7 CONCLUSION

The testing phase confirmed that all 21 user stories are fully implemented and validated. The system handles edge cases gracefully, enforces role-based access at every endpoint, and responds within acceptable performance thresholds. No critical or high-severity defects remain.

Chapitre

6 DISCUSSION AND EVALUATION

Sommaire

6.1 INTRODUCTION

This chapter evaluates the SIPMS platform against the initial objectives defined at the project outset. We discuss the strengths and limitations of the system, analyze the chosen technologies, and propose future enhancements.

6.2 EVALUATION OF OBJECTIVES

Objective	Status	Achievement
Multi-role authentication with RBAC	Achieved	JWT + Spring Security
Candidate registration and file management	Achieved	Reception Panel
Automated quiz assignment and grading	Achieved	Quiz System
AI-powered project ranking	Achieved	Spring AI Integration
Interview scheduling and result tracking	Achieved	Interview Module
Email notifications	Achieved	JavaMail + Gmail SMTP
Supervisor assignment and capacity management	Achieved	Supervisor Module

TABLE 6.1 – Evaluation of project objectives

6.3 TECHNOLOGY EVALUATION

6.3.1 Backend : Spring Boot 3.2 + Java 17

Spring Boot provided a robust foundation with built-in security, JPA integration, and REST API support. The layered architecture (Controller $\rightarrow$ Service $\rightarrow$ Repository) ensured clear separation of concerns and testability.

6.3.2 Frontend : React 19 + Vite

React's component-based model enabled rapid UI development. Vite's fast build times and HMR significantly improved development productivity. Tailwind CSS allowed consistent styling without writing custom CSS.

6.3.3 Database : MySQL 8

MySQL proved reliable for storing structured relational data. Hibernate's DDL auto-update simplified schema evolution during development.

6.3.4 AI Integration

Spring AI provided seamless integration with GPT-4-mini for project ranking and supervisor matching. The AI module operates as a recommendation engine rather than a decision maker, preserving human oversight.

6.4 LIMITATIONS

- **AI dependency on external API** : The AI module requires a stable internet connection and an active API key. Offline fallback is not implemented.
- **Email delivery reliability** : Gmail SMTP has daily send limits that could be reached in high-volume deployments.
- **File storage** : Currently uses local filesystem storage. A cloud storage solution (AWS S3, MinIO) would be more scalable.
- **Real-time notifications** : The system uses polling instead of WebSockets for in-app notifications, which may introduce slight delays.

6.5 FUTURE ENHANCEMENTS

- **WebSocket integration** for real-time notification delivery

- **Cloud file storage** with AWS S3 or MinIO for better scalability
- **Multi-language support** for international internship programs
- **Advanced analytics dashboard** with charts and exportable reports
- **Mobile application** using React Native for on-the-go access
- **Calendar integration** with Google Calendar / Outlook for interview scheduling

6.6 CONCLUSION

The SIPMS platform successfully meets all core requirements defined at the start of the project. While some limitations exist, the system is production-ready and deployed at CLINISYS. The modular architecture allows easy addition of future enhancements.

GENERAL CONCLUSION

This final-year project resulted in the successful design and development of **SIPMS (Smart Internship and Project Management System)**, a comprehensive full-stack web platform built for **CLINISYS** to digitize and automate the complete internship management lifecycle.

Throughout this project, we addressed a real organizational need : replacing a fragmented, manual internship management workflow with a structured, secure, and intelligent digital platform. The system was built following the **Scrum agile methodology**, delivering four functional sprints that progressively implemented all required features.

The key achievements of this project are :

- **Secure multi-role authentication** : A stateless JWT-based system with BCrypt password hashing and fine-grained RBAC enforced across all API endpoints.
- **Automated candidate onboarding** : The Reception Panel allows receptionists to register candidates, upload multiple documents, filter by internship year, and trigger automated welcome emails — all within a single workflow.
- **Objective competency evaluation** : A fully configurable quiz system with a timed interface, automatic scoring, and status management ensures fair and consistent candidate assessment.
- **AI-powered decision support** : Project ranking and candidate-to-supervisor matching algorithms provide managers with data-driven recommendations, reducing subjectivity in assignment decisions.
- **Complete application lifecycle** : From physical reception to final supervised project assignment, every stage of the internship process is tracked, versioned, and auditable.

From a technical perspective, this project provided valuable hands-on experience with a modern, production-grade technology stack : **React (Vite)** for a reactive and accessible frontend, **Spring Boot (Java 17)** for a robust and scalable backend, **MySQL** for relational data management,

and **Spring Security + JWT** for enterprise-grade authentication. The integration of **JavaMail** for automated notifications and **Spring Multipart** for document management further enriched the technical scope of the project.

Looking ahead, several enhancements could further strengthen the SIPMS platform :

- **Mobile application** : A React Native or Flutter companion app allowing candidates to track their status and receive push notifications on the go.
- **Advanced AI models** : Replacing the current scoring engine with machine learning models trained on historical internship outcome data to produce more accurate matchings.
- **Electronic signature** : Integrating a digital contract signing module to fully dematerialize the internship agreement workflow.
- **Analytics dashboard** : A management reporting module with visual KPIs tracking acceptance rates, quiz performance distributions, and supervisor workload trends.
- **Docker containerization** : Packaging the entire stack (frontend, backend, database) into Docker Compose for simplified deployment and horizontal scalability.

In conclusion, the SIPMS platform successfully delivers on its core objective : transforming a manual, error-prone process into an efficient, transparent, and intelligent system that benefits all stakeholders — from the receptionist registering a candidate on day one, to the manager making a data-informed project assignment decision weeks later. This project stands as a solid foundation for continued digital innovation within CLINISYS.

Bibliographie

[1] Pivotal Software, *Spring Boot Reference Documentation*, version 3.2, Available at : https://docs.spring.io/spring-boot/docs/current/reference/html/, 2024.

[2] Pivotal Software, *Spring Security Reference*, version 6.x, Available at : https://docs.spring.io/spring-security/reference/, 2024.

[3] Meta Platforms Inc., *React Documentation*, Available at : https://react.dev/, 2024.

[4] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[5] Evan You, *Vite — Next Generation Frontend Tooling*, Available at : https://vitejs.dev/, 2024.

[6] Oracle Corporation, *MySQL 8.0 Reference Manual*, Available at : https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[7] Red Hat Inc., *Hibernate ORM Documentation*, version 6.x, Available at : https://hibernate.org/orm/documentation/, 2024.

[8] Oracle Corporation, *JavaMail API Documentation*, Available at : https://javaee.github.io/javamail/, 2023.

[9] N. Provos, D. Mazières, *A Future-Adaptable Password Scheme*, Proceedings of USENIX Annual Technical Conference, 1999.

[10] K. Schwaber, J. Sutherland, *The Scrum Guide — The Definitive Guide to Scrum : The Rules of the Game*, November 2020, Available at : https://scrumguides.org/.

[11] Object Management Group (OMG), *Unified Modeling Language (UML) Specification*, version 2.5.1, Available at : https://www.omg.org/spec/UML/, 2017.

[12] R. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, PhD Dissertation, University of California, Irvine, 2000.

[13] Lucide Contributors, *Lucide React — Beautiful & consistent icon toolkit*, Available at : https://lucide.dev/, 2024.

ANNEXES

A.1 TITRE ANNEXE 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim.

FIGURE A.1 – Titre de figure (exemple)

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Institut International de Technologie de
Sfax

وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة للتكنولوجيا بصفا قس

TITRE DU PROJET

Prénom NOM

Résumé :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Mots clés : Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.

Abstract :

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim. Pellentesque congue.

Key-words : Lorem, ipsum, dolor, sit amet, consectetur, adipiscing elit.